Quiz 3 — Practice Problems

Reference: Banda-Barragán, W. E. (2026). *Computational Physics 1: Unit 4 lecture notes.* GitHub. https://github.com/wbandabarragan/computational-physics-1/tree/main/unit-4

How to work through it

- **The quiz will be written, and taken without a computer, a notebook or a phone.** You will answer concept questions, hand-write pseudocode, and sketch workflow diagrams.
- **Understand the math by hand.** The forward, backward and central differences, the Riemann sums, the trapezoid rule and Simpson's rule, each with the order of its error. You should be able to write them down and indicate their order of accuracy.
- **Practise writing pseudocode by hand.** Numbered steps, one action per line, the loops and the conditionals written out explicitly, and every function named with inputs and outputs. The logic has to be complete.
- Run each notebook of Unit 4, then understand and replicate the code. Practise coding!
- Submission of these problems is not needed. This sheet is practice for Quiz 3.

Part A — What to study

- **401 — Numerical differentiation.** Analytical derivatives with symbolic algebra; the forward, backward and central finite differences and their order of accuracy; derivatives from shifted arrays, and the mid-points at which they are defined.
- **402 — Errors in numerical differentiation.** Truncation error against round-off error, the maximum error, the log-log error plot and the meaning of its slope, and what happens when h is made too small.
- **403 — Derivatives with NumPy.** How `np.diff` and `np.gradient` work and how they differ; the derivative of a time series; smoothing by interpolation before differentiating, and why that is sometimes needed.
- **404 - 405 — Derivatives on 1D data.** Reading data files; calculating derivatives of discrete data by the forward and the central difference methods; assessing derivative errors. Different norms.
- **406 — Derivatives in ND.** Reading meshes and arrays; `np.gradient` on a 2D array and its components; the gradient of a scalar field, its modulus and edge detection by thresholding; the curl and the divergence of vector fields; checking that the units come out right.
- **407 — Numerical integration.** Symbolic integration; the left, right and midpoint Riemann integrals; the trapezoid rule; Simpson's rule. How errors scale in each method.
- **408 — Numerical integration with SciPy.** When to use `trapezoid`, `simpson`; `quad` functions; double and triple integrals; integrals to $\pm\infty$ with symbolic algebra.
- **409 — Integrals on data.** Reading data and calculating integrals on discrete data; the integral under a measured curve by the trapezoid and the Simpson methods.
- **410 — Integrals in ND.** Reading a mesh and its arrays, slicing, projecting and plotting planes; integrated quantities over the grid, the volume element dV; the centre of mass, and the centre of mass of a tagged region, etc.

Part B — Pseudocode and Workflow Diagrams

Write pseudocode for each of the problems below, and beside it a workflow diagram using the symbols of Lecture 107. Mark the data I/O, the operations, the decisions and the loops, and name every function with its inputs and its outputs.

1. **Choosing the step size.** You have a function whose derivative you also know analytically. Write the pseudocode of a program that sweeps h over several decades, evaluates the forward and the central difference at a fixed point, records the absolute error of each, and locates the h at which the error stops falling. Say explicitly what sets that minimum.

2. **A derivative from measured data.** A file holds two columns, time and position, sampled at a uniform Δt, with a few values missing. Write the pseudocode of a program that reads the file, fills the gaps by interpolation, computes the velocity with central differences in the interior and one-sided differences at the two ends, and plots position and velocity on the same time axis.

3. **An integral under a measured curve.** A file holds a rate $\dot{m}(t)$ sampled at uneven times. Write the pseudocode of a program that integrates it with the trapezoid rule, re-integrates it with Simpson's rule after resampling onto an even grid, compares the two totals, and plots the running integral against time.

4. **Mass and centre of mass on a grid.** A 3D mesh holds the density $\rho(x, y, z)$, a tracer $C(x, y, z)$ that marks one component of the gas, and the cell sizes. Write the pseudocode of a program that builds the volume element, returns the total mass and the centre of mass of the tagged component, and reports what changes if dV is left out.